

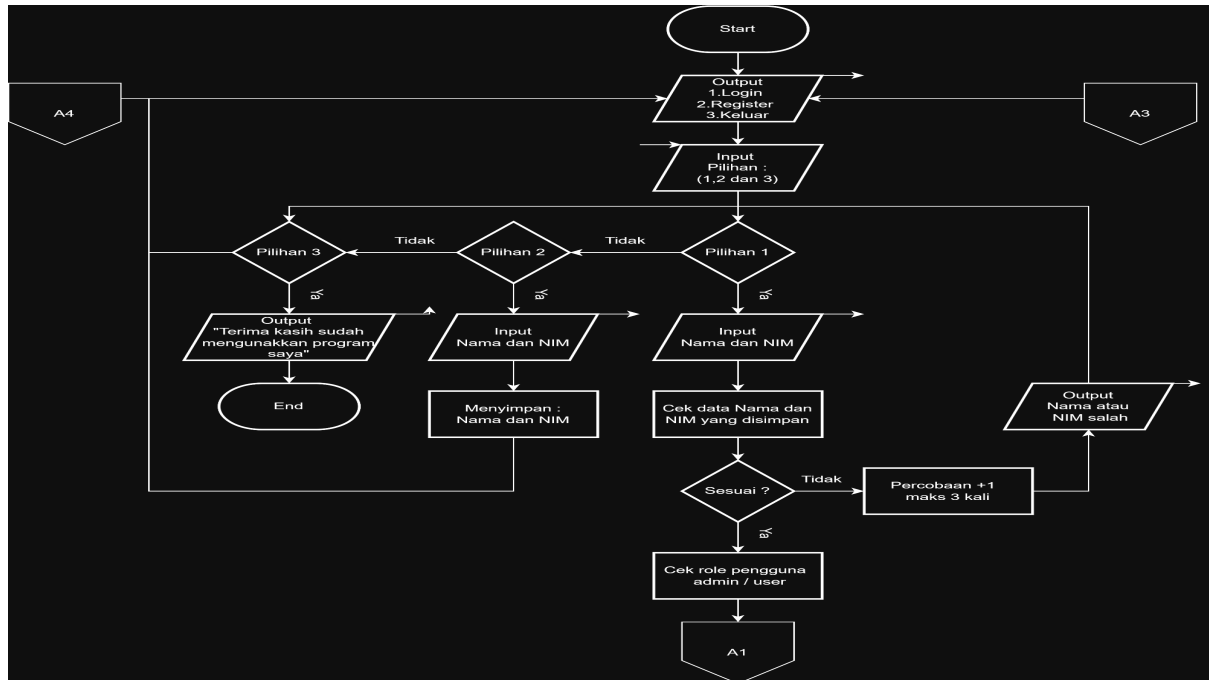
LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGRAMAN LANJUT



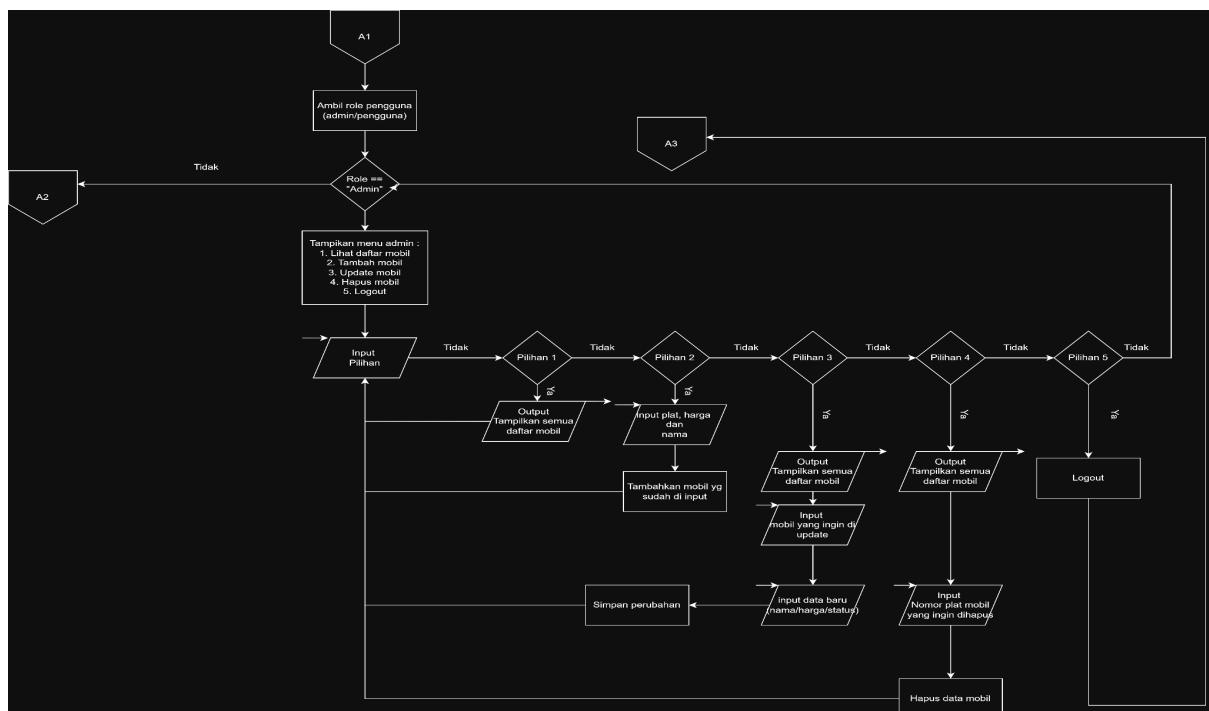
Disusun oleh:
Ahmad Aril Fadillah.B 2509106119
Informatika C2 25

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2026

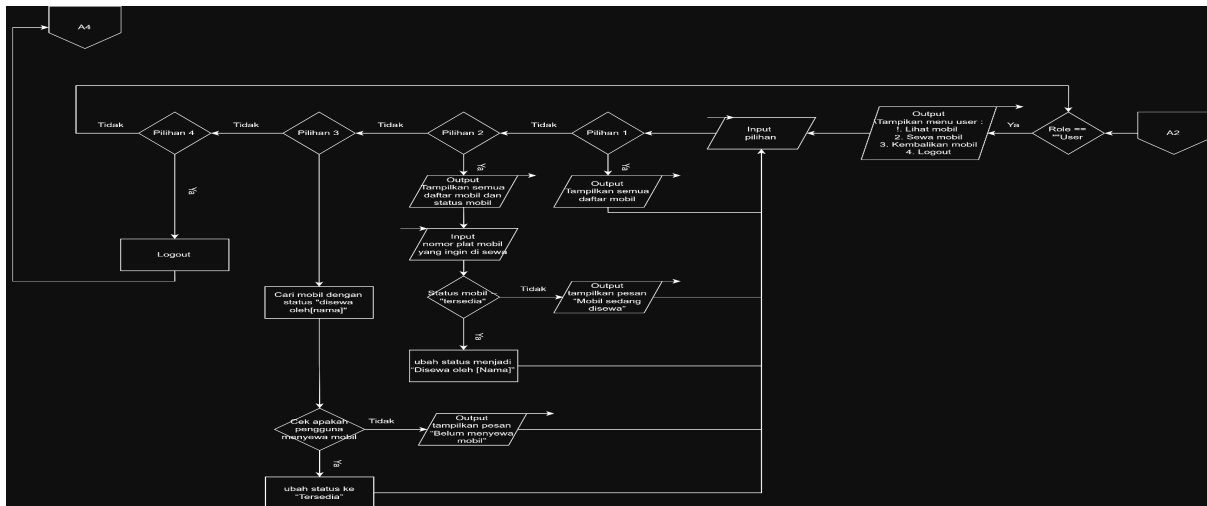
1. Flowchart



Gambar 1.1 Flowchart



Gambar 1.2 Flowchart



Gambar 1.2 Flowchart

PENJELASAN :

1. Start (Mulai)

Program dimulai dari Start. Setelah itu sistem akan menampilkan menu utama kepada pengguna.

Menu yang ditampilkan terdiri dari:

- Login
- Register
- Keluar

Pengguna diminta untuk memasukkan pilihan berupa angka 1, 2, atau 3.

2. Pilihan Menu Awal

Pilihan 1 = Login

Jika pengguna memilih Login, maka sistem akan meminta pengguna menginput Nama dan NIM. Sistem kemudian akan mengecek apakah data tersebut sesuai dengan data yang tersimpan.

Jika data tidak sesuai:

- Sistem menampilkan pesan bahwa nama atau NIM salah
- Percobaan login bertambah
- Maksimal percobaan adalah 3 kali
- Jika sudah 3 kali gagal, maka program akan berhenti

Jika data sesuai:

- Sistem akan mengecek role pengguna, apakah sebagai Admin atau User
- Setelah itu pengguna akan diarahkan ke menu sesuai rolenya

Pilihan 2 = Register

Jika pengguna memilih Register:

- Pengguna diminta memasukkan Nama dan NIM
- Data tersebut disimpan ke dalam sistem sebagai user baru
- Setelah itu pengguna dapat login menggunakan data yang sudah dibuat
- Program kembali ke menu utama

Pilihan 3 = Keluar

Jika pengguna memilih Keluar:

- Sistem menampilkan pesan “Terima kasih sudah menggunakan program”
- Program berakhir dan masuk ke End (Selesai)

3. Menu Admin

Jika pengguna yang login memiliki role sebagai Admin, maka sistem akan menampilkan menu admin.

Menu admin terdiri dari:

- Lihat daftar mobil
- Tambah mobil
- Update mobil
- Hapus mobil
- Logout

Penjelasan setiap menu:

1. Lihat daftar mobil

Sistem menampilkan seluruh data mobil yang ada di dalam sistem.

2. Tambah mobil

Admin diminta menginput:

- Nomor plat mobil
- Nama mobil
- Harga sewa

Setelah itu mobil akan ditambahkan ke dalam daftar.

3. Update mobil

- Sistem menampilkan daftar mobil
- Admin memilih mobil berdasarkan nomor plat
- Admin mengubah data seperti nama mobil, harga, dan status
- Sistem menyimpan perubahan tersebut

4. Hapus mobil

- Sistem menampilkan daftar mobil
- Admin memasukkan nomor plat mobil yang ingin dihapus
- Sistem menghapus data mobil dari daftar

5. Logout

- Admin keluar dari sistem
- Program kembali ke menu utama

4. Menu User

Jika pengguna yang login adalah User, maka sistem akan menampilkan menu user.

Menu user terdiri dari:

- Lihat mobil
- Sewa mobil
- Kembalikan mobil
- Logout

Penjelasan setiap menu:

1. Lihat mobil

Sistem menampilkan daftar mobil yang tersedia beserta statusnya.

2. Sewa mobil

- Sistem menampilkan daftar mobil
- User memasukkan nomor plat mobil yang ingin disewa
- Sistem mengecek status mobil

Jika mobil tidak tersedia:

- Sistem menampilkan pesan bahwa mobil sedang disewa

Jika mobil tersedia:

- Status mobil diubah menjadi “Disewa”
- Data penyewa disimpan

3. Kembalikan mobil

- Sistem mengecek apakah user sedang menyewa mobil

Jika tidak:

- Sistem menampilkan pesan bahwa user belum menyewa mobil

Jika iya:

- Status mobil diubah kembali menjadi “Tersedia”
- Data penyewa dihapus

4. Logout

- User keluar dari sistem
- Program kembali ke menu utama

5. End (Selesai)

Program akan berakhir jika pengguna memilih menu keluar atau gagal login sebanyak 3 kali dan p\rogram akan berakhir jika pengguna memilih Keluar pada menu utama.

2. Deskripsi Singkat Program

Program ini dibuat untuk mengelola penyewaan mobil secara sederhana. Dengan program ini, proses melihat mobil, menyewa mobil, dan mengembalikan mobil bisa dilakukan melalui sistem di komputer.

Di dalam program ada dua peran yaitu admin dan user.

Admin bertugas untuk mengelola data mobil, seperti melihat daftar mobil, menambah mobil baru, mengubah data mobil, dan menghapus mobil. Sedangkan user dapat melihat mobil yang tersedia, menyewa mobil, dan mengembalikan mobil yang sudah disewa.

Manfaat dari program ini adalah supaya pengelolaan data mobil dan proses penyewaan menjadi lebih mudah, rapi, dan teratur, sehingga tidak perlu dilakukan secara manual

3. Source Code

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <algorithm>
#include <string>
#include <vector>
#include <stdexcept>
#include <ctime>
#include <limits>

using namespace std;

namespace RentalSystem {

    struct User {
        string nama;
        string nim;
        string role;
    };

    struct Penyewa {
        string nama;
        string nim;
    };

    struct Mobil {
        string plat;
        string nama;
        int harga;
        string status;
        Penyewa penyewa;
        Penyewa *ptrPenyewa;
    };

    void clear() {
        #ifdef _WIN32
            system("cls");
        #else
            system("clear");
        #endif
    }
}
```

```

}

string getCurrentTime() {
    time_t now = time(0);
    char* dt = ctime(&now);
    return string(dt);
}

void clearBuffer() {
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}

void pause() {
    cout << "\nTekan Enter untuk melanjutkan...";
    clearBuffer();
    cin.get();
}

int hitungMobil(int n) {
    if(n==0) return 0;
    return 1 + hitungMobil(n-1);
}

void urutkanNamaMobilDesc(Mobil *mobil, int jumlahMobil) {
    for (int i = 0; i < jumlahMobil - 1; i++) {
        for (int j = 0; j < jumlahMobil - i - 1; j++) {
            if ((mobil + j)->nama < (mobil + j + 1)->nama) {
                Mobil temp = *(mobil + j);
                *(mobil + j) = *(mobil + j + 1);
                *(mobil + j + 1) = temp;
            }
        }
    }
}

void urutkanHargaMobilAsc(Mobil *mobil, int jumlahMobil) {
    for (int i = 0; i < jumlahMobil - 1; i++) {
        for (int j = 0; j < jumlahMobil - i - 1; j++) {
            if ((mobil + j)->harga > (mobil + j + 1)->harga) {
                Mobil temp = *(mobil + j);
                *(mobil + j) = *(mobil + j + 1);
                *(mobil + j + 1) = temp;
            }
        }
    }
}

```



```

    }
}

void urutkanPlatMobilAsc(Mobil *mobil, int jumlahMobil) {
    for (int i = 0; i < jumlahMobil - 1; i++) {
        for (int j = 0; j < jumlahMobil - i - 1; j++) {
            if ((mobil + j)->plat > (mobil + j + 1)->plat) {
                Mobil temp = *(mobil + j);
                *(mobil + j) = *(mobil + j + 1);
                *(mobil + j + 1) = temp;
            }
        }
    }
}

void tampilkanSatuMobil(Mobil m) {
    cout << left << setw(12) << m.plat
        << setw(25) << m.nama
        << setw(12) << m.harga
        << setw(15) << m.status << endl;
}

void lihatMobil(Mobil *mobil, int jumlahMobil) {
    clear();
    cout << "\n===== " << endl;
    cout << "          DAFTAR MOBIL          =" << endl;
    cout << "    " << getCurrentTime().substr(0, 19) << "    =" << endl;
    cout << "===== " << endl;
    cout << left << setw(12) << "Plat" << setw(25) << "Nama Mobil" << setw(12) << "Harga" << setw(15) <<
"Status" << endl;
    cout << "===== " << endl;
    for(int i=0; i<jumlahMobil; i++) tampilkanSatuMobil(*(mobil+i));
    cout << "===== " << endl;
    cout << "Total Mobil : " << hitungMobil(jumlahMobil) << endl;
    pause();
}

void lihatMobil(Mobil *mobil, int jumlahMobil, string status) {
    clear();
    cout << "\n===== " << endl;
    cout << "          DAFTAR MOBIL (" << status << ")          =" << endl;
    cout << "===== " << endl;

```

```

    cout << left << setw(12) << "Plat" << setw(25) << "Nama Mobil" << setw(12) << "Harga" << setw(15) <<
"Status" << endl;
    cout << "===== " << endl;

    bool found = false;
    for(int i=0; i<jumlahMobil; i++){
        if((mobil+i)->status == status){
            tampilkanSatuMobil(*(mobil+i));
            found = true;
        }
    }
    if(!found) cout << "Tidak ada mobil dengan status tersebut." << endl;
    cout << "===== " << endl;
    pause();
}

void lihatMobilAdminSorted(Mobil *mobil, int &jumlahMobil) {
    int pilihSort;
    do {
        clear();
        cout << "\n===== " << endl;
        cout << "          MENU SORTING ADMIN          =" << endl;
        cout << "===== " << endl;
        cout << "1. Lihat Semua (Tanpa Urutan Khusus)" << endl;
        cout << "2. Urutkan Nama Mobil (Descending / Z-A)" << endl;
        cout << "3. Urutkan Harga Mobil (Ascending / Murah-Mahal)" << endl;
        cout << "4. Kembali ke Menu Utama" << endl;
        cout << "Pilih : ";

        if (!(cin >> pilihSort)) { clearBuffer(); continue; }

        if(pilihSort == 1) {}
        else if(pilihSort == 2) { urutkanNamaMobilDesc(mobil, jumlahMobil); cout << "\n[INFO] Diurutkan
Nama (Z-A)\n"; }
        else if(pilihSort == 3) { urutkanHargaMobilAsc(mobil, jumlahMobil); cout << "\n[INFO] Diurutkan
Harga (Murah-Mahal)\n"; }

        if(pilihSort >= 1 && pilihSort <= 3) {
            cout << "\n===== " << endl;
            cout << left << setw(12) << "Plat" << setw(25) << "Nama Mobil" << setw(12) << "Harga" <<
setw(15) << "Status" << endl;
            cout << "===== " << endl;
            for(int i=0; i<jumlahMobil; i++) tampilkanSatuMobil(*(mobil+i));
            cout << "===== " << endl;

```

```

        cout << "Total Mobil : " << jumlahMobil << endl;
        pause();
    }
} while(pilihSort != 4);
}

void cariMobilFitur(Mobil *mobil, int jumlahMobil) {
    clear();
    int tipeSearch;
    cout << "\n===== " << endl;
    cout << "      PENCARIAN MOBIL      " << endl;
    cout << "===== " << endl;
    cout << "1. Cari berdasarkan NAMA (Linear Search)" << endl;
    cout << "2. Cari berdasarkan PLAT (Binary Search)" << endl;
    cout << "Pilih Metode: ";

    if (!(cin >> tipeSearch)) { clearBuffer(); cout << "Input tidak valid."; pause(); return; }
    clearBuffer();

    try {
        if (tipeSearch == 1) {
            string keyword;
            cout << "Masukkan kata kunci nama mobil : ";
            getline(cin, keyword);
            if(keyword.empty()) throw invalid_argument("Kata kunci tidak boleh kosong");

            cout << "\nHasil Pencarian Linear:\n";
            cout << "===== " << endl;
            cout << left << setw(12) << "Plat" << setw(25) << "Nama Mobil" << setw(12) << "Harga" <<
setw(15) << "Status" << endl;
            cout << "===== " << endl;
            bool foundAny = false;
            for(int i=0; i<jumlahMobil; i++){
                if((mobil+i)->nama.find(keyword) != string::npos){
                    tampilkanSatuMobil(*(mobil+i));
                    foundAny = true;
                }
            }
            if(!foundAny) throw runtime_error("Mobil tidak ditemukan.");
        } else if (tipeSearch == 2) {
            string targetPlat;
            cout << "Masukkan Plat Nomor tepat : ";
            getline(cin, targetPlat);
            if(targetPlat.empty()) throw invalid_argument("Plat nomor tidak boleh kosong");

```

```

        urutkanPlatMobilAsc(mobil, jumlahMobil);
        int left = 0, right = jumlahMobil - 1, index = -1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if ((mobil + mid)->plat == targetPlat) { index = mid; break; }
            if ((mobil + mid)->plat < targetPlat) left = mid + 1;
            else right = mid - 1;
        }

        cout << "\n===== " << endl;
        cout << left << setw(12) << "Plat" << setw(25) << "Nama Mobil" << setw(12) << "Harga" <<
        setw(15) << "Status" << endl;
        cout << "===== " << endl;
        if(index != -1) tampilkanSatuMobil(*(mobil + index));
        else throw runtime_error("Mobil dengan plat tersebut TIDAK DITEMUKAN.");
    } else throw invalid_argument("Pilihan tidak valid.");
} catch (const exception& e) {
    cout << "\n[ERROR] " << e.what() << endl;
}
pause();
}

void tambahMobil(Mobil *mobil, int &jumlahMobil, int maxMobil) {
    clear();
    if (jumlahMobil >= maxMobil) throw overflow_error("Kapasitas garasi penuh!");

    cout << "\n===== " << endl;
    cout << "      Tambah Mobil      " << endl;
    cout << "===== " << endl;

    string platBaru;
    cout << "Plat : "; getline(cin, platBaru);
    for(int i=0; i<jumlahMobil; i++) if((mobil+i)->plat == platBaru) throw invalid_argument("Plat sudah
    terdaftar!");
    (mobil+jumlahMobil)->plat = platBaru;

    cout << "Nama Mobil : "; getline(cin, (mobil+jumlahMobil)->nama);
    if((mobil+jumlahMobil)->nama.empty()) throw invalid_argument("Nama mobil tidak boleh kosong");

    cout << "Harga : ";
    if (!(cin >> (mobil+jumlahMobil)->harga) || (mobil+jumlahMobil)->harga <= 0) throw
    invalid_argument("Harga harus angka positif!");
}

```

```

(mobil+jumlahMobil)->status = "Tersedia";
(mobil+jumlahMobil)->ptrPenyewa = &((mobil+jumlahMobil)->penyewa);
(mobil+jumlahMobil)->penyewa = {"", ""};
jumlahMobil++;
cout << "\n[SUCCESS] Mobil berhasil ditambahkan.";
pause();
}

void updateMobil(Mobil *mobil, int jumlahMobil) {
    clear();
    cout << "\n===== " << endl;
    for(int i=0; i<jumlahMobil; i++) cout << (i+1) << ". " << (mobil+i)->plat << " - " << (mobil+i)->nama <<
endl;

    string plat;
    cout << "\nMasukkan plat mobil yang mau diupdate : "; cin >> plat;
    for(int i=0; i<jumlahMobil; i++){
        if((mobil+i)->plat == plat){
            clearBuffer();
            cout << "Nama baru : "; getline(cin, (mobil+i)->nama);
            cout << "Harga baru : ";
            if(!(cin >> (mobil+i)->harga) || (mobil+i)->harga <= 0) throw invalid_argument("Harga invalid");
            cout << "Status (Tersedia/Disewa): "; cin >> (mobil+i)->status;
            if((mobil+i)->status != "Tersedia" && (mobil+i)->status != "Disewa") throw
invalid_argument("Status invalid");
            cout << "\n[SUCCESS] Data berhasil diupdate.";
            pause(); return;
        }
    }
    throw runtime_error("Mobil tidak ditemukan");
}

void hapusMobil(Mobil *mobil, int &jumlahMobil) {
    clear();
    cout << "\n===== " << endl;
    for(int i=0; i<jumlahMobil; i++) cout << (i+1) << ". " << (mobil+i)->plat << " - " << (mobil+i)->nama <<
endl;

    string plat;
    cout << "\nMasukkan plat mobil yang mau dihapus : "; cin >> plat;
    for(int i=0; i<jumlahMobil; i++){
        if((mobil+i)->plat == plat){
            if((mobil+i)->status == "Disewa") throw runtime_error("Tidak bisa menghapus mobil yang sedang
disewa!");

```

```

        for(int j=i; j<jumlahMobil-1; j++) *(mobil+j) = *(mobil+j+1);
        jumlahMobil--;
        cout << "\n[SUCCESS] Mobil berhasil dihapus.";
        pause(); return;
    }
}

throw runtime_error("Mobil tidak ditemukan");
}

void sewaMobil(Mobil *mobil, int jumlahMobil, string nama, string nim) {
    clear();
    for(int i=0; i<jumlahMobil; i++)
        if((mobil+i)->ptrPenyewa->nama == nama && (mobil+i)->status == "Disewa")
            throw runtime_error("Anda masih menyewa mobil " + string((mobil+i)->nama) + ". Kembalikan
dulu.");

    cout << "\n===== " << endl;
    cout << "          Mobil Tersedia    \n";
    cout << "===== " << endl;
    cout << left << setw(12) << "Plat" << setw(25) << "Nama Mobil" << setw(12) << "Harga" << setw(15) <<
"Status" << endl;
    bool ada = false;
    for(int i=0; i<jumlahMobil; i++){
        if((mobil+i)->status == "Tersedia"){ tampilkanSatuMobil(*(mobil+i)); ada = true; }
    }
    if(!ada) throw runtime_error("Tidak ada mobil tersedia.");

    string plat;
    cout << "\nMasukkan plat mobil yang ingin disewa : "; cin >> plat;
    for(int i=0; i<jumlahMobil; i++){
        if((mobil+i)->plat == plat){
            if((mobil+i)->status == "Tersedia"){
                (mobil+i)->status = "Disewa";
                (mobil+i)->ptrPenyewa->nama = nama;
                (mobil+i)->ptrPenyewa->nim = nim;
                cout << "\n[SUCCESS] Mobil berhasil disewa.";
            } else throw runtime_error("Mobil sedang disewa orang lain.");
            pause(); return;
        }
    }
    throw runtime_error("Plat tidak ditemukan");
}

void kembalikanMobil(Mobil *mobil, int jumlahMobil, string nama) {

```

```

clear();
for(int i=0; i<jumlahMobil; i++){
    if((mobil+i)->ptrPenyewa->nama == nama && (mobil+i)->status == "Disewa"){
        (mobil+i)->status = "Tersedia";
        (mobil+i)->ptrPenyewa->nama = ""; (mobil+i)->ptrPenyewa->nim = "";
        cout << "\n[SUCCESS] Mobil " << (mobil+i)->nama << " berhasil dikembalikan.";
        pause(); return;
    }
}
throw runtime_error("Anda tidak sedang menyewa mobil apapun.");
}

void menuAdmin(Mobil *mobil, int &jumlahMobil, int maxMobil) {
    int pilih;
    do {
        clear();
        cout << "\n===== " << endl;
        cout << "          MENU ADMIN          " << endl;
        cout << "===== " << endl;
        cout << "1. Lihat & Urutkan Mobil\n2. Tambah Mobil\n3. Update Mobil\n4. Hapus Mobil\n5. Cari\n6. Logout\n";
        cout << "Pilih : ";
        if(!(cin >> pilih)) { clearBuffer(); continue; }

        try {
            if(pilih==1) lihatMobilAdminSorted(mobil, jumlahMobil);
            else if(pilih==2) tambahMobil(mobil, jumlahMobil, maxMobil);
            else if(pilih==3) updateMobil(mobil, jumlahMobil);
            else if(pilih==4) hapusMobil(mobil, jumlahMobil);
            else if(pilih==5) cariMobilFitur(mobil, jumlahMobil);
        } catch (const exception& e) {
            cout << "\n[EXCEPTION] " << e.what() << endl; pause();
        }
    } while(pilih != 6);
}

void menuUser(Mobil *mobil, int jumlahMobil, string nama, string nim) {
    int pilih;
    do {
        clear();
        cout << "\n===== " << endl;
        cout << "          MENU USER          " << endl;
        cout << "===== " << endl;
        cout << "1. Lihat Mobil Tersedia\n2. Sewa Mobil\n3. Kembalikan Mobil\n4. Cari Mobil\n5. Logout\n";
    }
}

```

```

        cout << "Pilih : ";
        if(!(cin >> pilih)) { clearBuffer(); continue; }

        try {
            if(pilih==1) lihatMobil(mobil, jumlahMobil, "Tersedia");
            else if(pilih==2) sewaMobil(mobil, jumlahMobil, nama, nim);
            else if(pilih==3) kembalikanMobil(mobil, jumlahMobil, nama);
            else if(pilih==4) cariMobilFitur(mobil, jumlahMobil);
        } catch (const exception& e) {
            cout << "\n[EXCEPTION] " << e.what() << endl; pause();
        }
    } while(pilih != 5);
}

void registerUser(User users[], int &jumlahUser, int maxUser) {
    clear();
    if(jumlahUser >= maxUser) throw overflow_error("Database user penuh!");
    cout << "\n===== " << endl;
    cout << "          REGISTER          " << endl;
    cout << "===== " << endl;
    cout << "Nama : "; getline(cin, users[jumlahUser].nama);
    cout << "NIM : "; getline(cin, users[jumlahUser].nim);
    users[jumlahUser].role = "user";
    jumlahUser++;
    cout << "\n[SUCCESS] Register berhasil!"; pause();
}

void login(User users[], int jumlahUser, Mobil *mobil, int &jumlahMobil, int maxMobil) {
    string nama, nim;
    int percobaan = 0;
    while(percobaan < 3) {
        clear();
        cout << "\n===== " << endl;
        cout << "          LOGIN          " << endl;
        cout << "===== " << endl;
        cout << "Nama : "; getline(cin, nama);
        cout << "NIM : "; getline(cin, nim);
        for(int i=0; i<jumlahUser; i++) {
            if(users[i].nama == nama && users[i].nim == nim) {
                if(users[i].role == "admin") menuAdmin(mobil, jumlahMobil, maxMobil);
                else menuUser(mobil, jumlahMobil, nama, nim);
                return;
            }
        }
    }
}

```



```

        percobaan++;
        cout << "Login gagal! Sisa " << 3-percobaan << " kesempatan."; pause();
    }
    throw runtime_error("Login gagal 3 kali berturut-turut.");
}

}

int main() {
    using namespace RentalSystem;
    const int MAX_USER = 100, MAX_MOBIL = 100;
    User users[MAX_USER];
    Mobil mobil[MAX_MOBIL];
    int jumlahUser = 1, jumlahMobil = 5;

    users[0] = {"Aril", "119", "admin"};
    mobil[0] = {"KT1234AA", "Toyota Alphard", 1500000, "Tersedia", {"", ""}};
    mobil[1] = {"DA5678BB", "BMW i8", 2500000, "Tersedia", {"", ""}};
    mobil[2] = {"KH8765CC", "Mercedes Benz S-Class", 2800000, "Tersedia", {"", ""}};
    mobil[3] = {"KB4321DD", "Range Rover Evoque", 2000000, "Tersedia", {"", ""}};
    mobil[4] = {"KU9999EE", "Porsche Cayenne", 3000000, "Tersedia", {"", ""}};

    for(int i=0; i<jumlahMobil; i++) mobil[i].ptrPenyewa = &mobil[i].penyewa;

    int menu;
    do {
        clear();
        cout << "\n===== " << endl;
        cout << "    SISTEM RENTAL MOBIL    " << endl;
        cout << "    " << getCurrentTime().substr(0, 19) << "    " << endl;
        cout << "===== " << endl;
        cout << "1. Login\n2. Register\n3. Keluar\nPilih : ";
        if(!(cin >> menu)) { clearBuffer(); continue; }
        clearBuffer();

        try {
            if(menu==1) login(users, jumlahUser, mobil, jumlahMobil, MAX_MOBIL);
            else if(menu==2) { cout << "\n"; registerUser(users, jumlahUser, MAX_USER); }
        } catch (const exception& e) {
            string err = e.what();
            cout << "\n[FATAL] " << err << endl;
            if(err.find("Login gagal 3 kali") != string::npos) return 1;
            pause();
        }
    }
}

```

```

} while(menu != 3);

cout << "Terima kasih sudah menggunakan program saya !!\n";
return 0;
}

```

4. Hasil Output

```

=====
=          SISTEM RENTAL MOBIL          =
=      Tue Apr 28 18:04:32              =
=====
1. Login
2. Register
3. Keluar
Pilih : █

```

Gambar 4.1 Hasil Output

```

=====
=              REGISTER                  =
=====
Nama : Aril
NIM : 119
Register berhasil!
Tekan Enter...█

```

Gambar 4.2 Hasil Output

```

=====
=              MENU USER                 =
=====
1. Lihat Mobil Tersedia
2. Sewa Mobil
3. Kembalikan Mobil
4. Cari Mobil
5. Logout
Pilih : █

```

Gambar 4.3 Hasil Output

```
=====
=          MENU ADMIN          =
=====
1. Lihat & Urutkan Mobil
2. Tambah Mobil
3. Update Mobil
4. Hapus Mobil
5. Cari Mobil
6. Logout
Pilih : █
```

Gambar 4.4 Hasil Output

```
=====
=      SISTEM RENTAL MOBIL      =
=    Tue Apr 28 18:07:26        =
=====
1. Login
2. Register
3. Keluar
Pilih : 3
Terima kasih sudah menggunakan program saya !!
PS D:\praktikum-apl\post-test\post-test-apl-7\output> █
```

Gambar 4.1 Hasil Output

5. Langkah-langkah GIT

```
PS D:\praktikum-apl> git add .
PS D:\praktikum-apl> git commit -m "posttest 7"
[main 6b94573] posttest 7
11 files changed, 748 insertions(+)
create mode 100644 kelas/output/pertemuan-7...exe
create mode 100644 kelas/output/pertemuan-7..exe
create mode 100644 kelas/output/pertemuan-Graph & Optimisasi-7..exe
create mode 100644 kelas/output/petemuan-7.exe
create mode 100644 kelas/pertemuan-Brute force & Greedy Algorithm-7...cpp
create mode 100644 kelas/pertemuan-Brute force & Greedy Algorithm-7..cpp
create mode 100644 kelas/pertemuan-Brute force & Greedy Algorithm-7.cpp
create mode 100644 kelas/pertemuan-Graph & Optimisasi-7..cpp
create mode 100644 kelas/pertemuan-Graph & Optimisasi-7.cpp
create mode 100644 post-test/post-test-apl-7/2509106119-AhmadArilFadillah.B-PT-7.cpp
create mode 100644 post-test/post-test-apl-7/output/2509106119-AhmadArilFadillah.B-PT-7.exe
PS D:\praktikum-apl> git push
Enumerating objects: 22, done.
Counting objects: 100% (22/22), done.
Delta compression using up to 8 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (18/18), 1.96 MiB | 345.00 KiB/s, done.
Total 18 (delta 5), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (5/5), completed with 1 local object.
To https://github.com/ahmadarilfadilah1222-beep/praktikum-apl.git
1829c54..6b94573 main -> main
PS D:\praktikum-apl>
```

Gambar 5.1 Langkah-langkah git

1. Git init

```
PS D:\praktikum-apl> git init
Initialized empty Git repository in D:/praktikum-apl/.git/
```

Gambar 5.2 Langkah-langkah git

Git init adalah perintah untuk memulai atau menginisialisasi repository Git baru di dalam sebuah folder.

2. Git Add

```
PS D:\praktikum-apl> git add .
```

Gambar 5.3 Langkah-langkah git

Git add adalah perintah di Git untuk menambahkan perubahan file ke staging area (area persiapan) sebelum dilakukan commit.

3. Git commit

```
PS D:\praktikum-apl> git commit -m "first commit"
[main (root-commit) 6468cdc] first commit
2 files changed, 144 insertions(+)
create mode 100644 output/post-test/post-test-apl-1/250906119-AhmadArilFadillh.B-PT-1.exe
create mode 100644 post-test/post-test-apl-1/250906119-AhmadArilFadillh.B-PT-1.cpp
```

Gambar 5.4 Langkah-langkah git

Git commit tempat menyimpan suatu perubahan yg kita lakukan dengan menyimpan pesan

4. Git push

```
PS D:\praktikum-apl> git push -u origin main
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 8 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (9/9), 719.01 KiB | 2.61 MiB/s, done.
Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/ahmadarilfadilah1222-beep/praktikum-apl.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\praktikum-apl> 
```

Gambar 5.5 Langkah-langkah git

Git push adalah dimana kita mengirim sesuatu ke suatu tempat